Software Requirements Specifications for [Student Attendance System]

[15-Jan-2026]

Version [1.0]

Prepared By "Group 1"

1. **Tith Annchhengly**
2. **Thypheap Sachak Ponlue Pragna**
3. **Pa Soborith**
4. **Leang Serminh**
5. **Ty Kimhong**

Change History

Date	Version	Description	Updated By
[DD MMM YYYY]	[X.Y]	[e.g., Initial Draft / Added FR section / Updated NFR]	[Name]
[DD MMM YYYY]	[X.Y]	[e.g., Initial Draft / Added FR section / Updated NFR]	[Name]
[DD MMM YYYY]	[X.Y]	[e.g., Initial Draft / Added FR section / Updated NFR]	[Name]

Document Approvals

Name	Role	Signature
[Approver Name]	[Role: Product Owner / PM / Tech Lead / QA Lead]	
[Approver Name]	[Role: Product Owner / PM / Tech Lead / QA Lead]	
[Approver Name]	[Role: Product Owner / PM / Tech Lead / QA Lead]	
[Approver Name]	[Role: Product Owner / PM / Tech Lead / QA Lead]	

Table of Contents

Note: In Microsoft Word, right-click the table of contents and choose “Update Field” to refresh.

1. Introduction

1.1 Purpose of the document

[Describe the purpose of this SRS, who it is for, and how it will be used.]

1.2 Scope of the project

[Describe the system boundaries, major features, supported platforms, and what is out of scope.]

1.3 Stakeholders involved

1.3.1 Management / Product Owner

[Describe stakeholder responsibilities, interests, and inputs.]

1.3.2 Employees / Administrators

[Describe stakeholder responsibilities, interests, and inputs.]

1.3.3 Customers / End Users

[Describe stakeholder responsibilities, interests, and inputs.]

1.3.4 Third-Party Services (Payment/Shipping/etc.)

[Describe stakeholder responsibilities, interests, and inputs.]

1.3.5 Developers / QA / Operations

[Describe stakeholder responsibilities, interests, and inputs.]

1.4 Overview of the [PROJECT] portal/system

[Provide a brief overview of the product including major modules such as Catalog, Accounts, Cart, Payment, Fulfillment.]

2. Overall Description

2.1 Product perspective

[Describe how the product fits into the business environment; include integrations and external interfaces.]

2.2 Product functions

[List high-level functions for each user type. Use bullets.]

2.3 User characteristics

[Describe user types, skill levels, languages, accessibility needs, and security expectations.]

2.4 Constraints

[List constraints: technology, time, budget, legal/regulatory, performance/scalability, security.]

2.5 Assumptions and dependencies

[List assumptions and dependencies (infrastructure, third-party services, resources, timelines).]

2.6 Apportioning of requirements

[If phased delivery: Phase 1/2/3 with features per phase.]

3. Specific Requirements

3.1 Functional Requirements

3.1.1 Product browsing and searching

This functional area allows users to locate courses, student records, and leave requests to ensure efficient oversight of attendance data.

- **The system shall** provide a centralized search interface to locate students by name, ID, or RUPP email.
- **The system shall verify** user permissions (RBAC) before displaying detailed personal attendance or leave history.
- **The user shall be able to** filter attendance logs by date, course section, or status (Present, Absent, Late, or Approved Leave).
- **The system shall display** a real-time searchable roster of students currently checked into a specific session.
- **Example:** A teacher searches for "General English 101" to see the live attendance list and identifies which students are absent versus those with approved digital leave requests.

3.1.2 Product ordering and checkout

This area manages the "transaction" of submitting and processing leave requests digitally to replace paper forms.

- **The system shall** allow students to submit digital leave requests including dates and reasons.
- **The system shall validate** that a leave request is routed to the correct authority (Class Monitor, Teacher, or Head of Department) based on the RUPP process.
- **The user shall be able to** attach supporting documentation (e.g., medical certificates) to their digital request.
- **The system shall display** a confirmation screen once a request has been successfully submitted to the cloud backend.

- **Example:** A student submits a 2-day leave request via the app; the system automatically routes it to their Department Head for digital signature.

3.1.3 User account management

This area governs the roles and profiles required to access the integrated attendance and leave platform.

- **The system shall** require all users to register using an official RUPP email address.
- **The system shall verify** the selected role (Student, Teacher, or Admin/Head of Dept) during the account configuration phase.
- **The user shall be able to** update their profile information, including contact mobile numbers and notification preferences.
- **The system shall display** a "User Role" tag on the dashboard to indicate the current level of access and active status.
- **Example:** A new staff member signs up with their university email and selects the "Teacher" role to gain access to their specific class rosters

3.1.4 Payment processing

This area manages the core "value" of the system: accurately recording and computing attendance data.

- **The system shall** record digital check-in and check-out timestamps for every student session.
- **The system shall compute** total attendance percentages and identify students falling below the required academic threshold.
- **The user shall be able to** manually override an attendance status if a technical error occurs during digital check-in.
- **The system shall display** attendance data in visual formats such as bar charts or pie graphs on the dashboard.
- **Example:** At the end of the month, the system automatically calculates that a student has 85% attendance and generates a digital report for the administrator.

3.1.5 Order tracking and status

- This area ensures all parties are informed of attendance trends and leave request progress.

- **The system shall notify** students immediately via push notification or email when their leave request is approved or declined.
- **The system shall notify** teachers if a student's attendance drops significantly over a one-week period.
- **The user shall be able to** track the real-time status of a pending leave request as it moves through the approval chain.
- **The system shall display** a "History" log of all past check-ins and leave approvals for the current semester.
- **Example:** A student checks their app to see that their "Sick Leave" status has changed from "Pending" to "Approved" by the Department Head.

3.2 Non-functional Requirements

[Describe quality attributes: performance, usability, security, compatibility, availability, maintainability, etc.]

3.2.1 Performance requirements

- **Response time:** The system shall process digital check-ins in <= 2 seconds to prevent congestion at classroom entrances.

    - **Throughput:** The system shall support at least 500 concurrent users (students checking in simultaneously) during peak class start times.
    - **Scalability:** The system shall use AWS cloud hosting for horizontal scaling to handle university-wide growth.

3.2.2 Usability requirements

- **Navigation:** The interface shall feature a clear sidebar menu for easy switching between "Attendance," "Leave Requests," and "Reports".

    - **Search:** Search results for student records shall be returned in under 1 second.
    - **Mobile responsiveness:** The system shall be fully responsive for mobile check-ins and feature a "Light/Dark" mode for user preference.

-

3.2.3 Security requirements

- **Authentication:** Access shall be restricted to verified RUPP email accounts with secure password policies.

    - **Authorization:** The system shall implement Role-Based Access Control (RBAC) for Students, Teachers, and Department Heads.
    - **Encryption:** All data, especially biometric or personal student info, shall be encrypted in transit via HTTPS.

3.2.4 Compatibility requirements

- **Browsers:** The web dashboard shall support the latest versions of Chrome, Firefox, and Safari.
    - **Mobile:** The student interface shall support Android and iOS devices for mobile check-in.
    - **Database:** The system shall utilize a robust database (e.g., PostgreSQL via Django) to manage high volumes of attendance logs.
-

3.2.5 Availability requirements

- **Availability:** The cloud-based platform shall be available 24/7 to allow students to submit leave requests at any time.
    - **Uptime:** The system shall maintain 99.9% uptime during the active academic semester.
    - **Recovery:** In the event of a server failure, the system shall restore full functionality within 30 minutes using AWS backup services.

4. Use Cases

4.1 Product browsing and searching use case

Description: [What this use case is about.]

Actors: [Primary and secondary actors.]

Pre-conditions: [What must be true before starting.]

Post-conditions: [State after completion.]

Basic Flow: [Numbered steps for the normal path.]

Alternative Flows: [Variations / optional paths.]

Exceptions: [Error conditions / system failures.]

4.2 Product ordering and checkout use case

Description: [What this use case is about.]

Actors: [Primary and secondary actors.]

Pre-conditions: [What must be true before starting.]

Post-conditions: [State after completion.]

Basic Flow: [Numbered steps for the normal path.]

Alternative Flows: [Variations / optional paths.]

Exceptions: [Error conditions / system failures.]

4.3 User account management use case

Description: [What this use case is about.]

Actors: [Primary and secondary actors.]

Pre-conditions: [What must be true before starting.]

Post-conditions: [State after completion.]

Basic Flow: [Numbered steps for the normal path.]

Alternative Flows: [Variations / optional paths.]

Exceptions: [Error conditions / system failures.]

4.4 Payment processing use case

Description: [What this use case is about.]

Actors: [Primary and secondary actors.]

Pre-conditions: [What must be true before starting.]

Post-conditions: [State after completion.]

Basic Flow: [Numbered steps for the normal path.]

Alternative Flows: [Variations / optional paths.]

Exceptions: [Error conditions / system failures.]

4.5 Order tracking and status use case

Description: [What this use case is about.]

Actors: [Primary and secondary actors.]

Pre-conditions: [What must be true before starting.]

Post-conditions: [State after completion.]

Basic Flow: [Numbered steps for the normal path.]

Alternative Flows: [Variations / optional paths.]

Exceptions: [Error conditions / system failures.]

5. System Models

5.1 Class diagrams

[Insert class diagram(s) here. Provide captions and explain key entities/relationships.]

5.2 Sequence diagrams

[Insert sequence diagram(s) here. Explain lifelines and key messages.]

5.3 State diagrams

[Insert state diagram(s) here. Explain states and transitions.]

6. Appendices

6.1 Glossary of terms

[Add glossary items as bullets.]

- Term: [Definition]

6.2 List of acronyms and abbreviations

[Add acronyms as bullets.]

- ABC: [Meaning]

6.3 References and resources

[Add references/links.]

• [IEEE SRS / UML / OWASP / PCI DSS / etc.]